



TECNOLÓGICO
NACIONAL DE MÉXICO



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



INSTITUTO TECNOLÓGICO
DE CIUDAD MADERO

ITCM

INGENIERIA EN SISTEMAS COMPUTACIONALES
SIMULACIÓN

PRÁCTICA NO.6

**“IMPLEMENTACIÓN DE
PRUEBAS ESTADÍSTICAS”**

PT.1 DE 1

UNIDAD 1 | E1A | 12:00 a 3:00

ALUMNO

- Izaguirre Rodríguez Ana María

06 de Julio del 2026

Calificación: _____

Contenido

1	INSTRUCCIONES	3
2	NOTAS	3
2.1	PASO 1	3
2.1.1	Prueba de los Promedios	3
2.2	PASO 2	3
2.2.1	Prueba de Frecuencias	3
2.3	PASO 3	3
2.3.1	Prueba de Póquer	3
2.4	PASO 4	3
2.4.1	Prueba de las Corridas (Arriba y Abajo)	3
3	DESARROLLO	3
3.1	CÓDIGO	3
3.1.1	CÓDIGO CON NOTAS (CAPTURAS)	3
3.1.2	CÓDIGO SIN NOTAS (TEXTO)	7
3.2	EJECUCIÓN	16
3.2.1	RUN	16
3.2.2	ARCHIVO	17
4	CONCLUSIONES	17
5	GITHUB	17

1 INSTRUCCIONES

Con los números aleatorios obtenidos en la práctica del generador, implemente un programa que aplique las pruebas estadísticas para demostrar la independencia de los números pseudoaleatorios generados (una prueba de promedios, frecuencias, la de póquer y una prueba de corridas). Guarde en un archivo la semilla, las constantes y el módulo utilizados que pasen todas estas pruebas. Hacer un reporte y subir a Github, colocar aquí reporte y liga de Github.

2 NOTAS

2.1 [PASO 1](#)

2.1.1 Prueba de los Promedios

Evalúa si la media aritmética global de los 4096 valores se sitúa en la cercanía del valor central esperado de 0.5. Sirve como un filtro preliminar de equilibrio numérico básico.

2.2 [PASO 2](#)

2.2.1 Prueba de Frecuencias

Divide el espacio unidimensional en 5 subintervalos uniformes. Compara la cantidad de datos que caen en cada celda contra el valor esperado de 819.2 mediante el estadístico X^2 , validando la uniformidad multidimensional.

2.3 [PASO 3](#)

2.3.1 Prueba de Póquer

Analiza la composición combinatoria individual de los primeros 5 dígitos decimales de cada número generado, agrupando las estructuras en manos de juego (TD, 1P, 2P, T, FH, P, Q). Compara estas agrupaciones contra las densidades probabilísticas teóricas de eventos independientes, siendo la prueba más rigurosa para evaluar la dispersión aleatoria por dígitos.

2.4 [PASO 4](#)

2.4.1 Prueba de las Corridas (Arriba y Abajo)

Examina la secuencia binaria de tendencias de incremento y decremento de valores adyacentes para descartar la existencia de dependencias o patrones oscilatorios predecibles en el orden de generación de la serie.

3 DESARROLLO

3.1 [CÓDIGO](#)

3.1.1 CÓDIGO CON NOTAS (CAPTURAS)

```

package SIMULACION;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
public class SU1P6E1 {
public static void main(String[] args) {
    // 1. Parámetros de la Práctica 4 (Periodo Completo de 4096)

    int semilla = 7326;
    int a = 205;        // Nuevo multiplicador con alta entropía
    int b = 1399;       // Nueva constante aditiva prima relativa
    int M = 4096;

    List<Double> numeros = new ArrayList<>();
    int actual = semilla;

    // Generación de la serie de 4096 números normalizados entre 0 y 1
    for (int i = 0; i < M; i++) {
        int siguiente = (a * actual + b) % M;
        numeros.add((double) siguiente / M);
        actual = siguiente;
    }

    System.out.println("=====");
    System.out.println("==          PRÁCTICA 6: IMPLEMENTACIÓN DE PRUEBAS ESTADÍSTICAS MASIVAS          ==");
    System.out.println("=====");
    System.out.println("-> Procesando " + M + " números pseudoaleatorios generados...");

    // Ejecución de las 4 pruebas estadísticas requeridas
    boolean pasoPromedios = evaluarPromedios(numeros);
    boolean pasoFrecuencias = evaluarFrecuencias(numeros);
    boolean pasoPoquer = evaluarPoquer(numeros);
    boolean pasoCorridas = evaluarCorridas(numeros);

    System.out.println("\n=====");
    System.out.println("==          RESUMEN DE AUDITORÍA          ==");
    System.out.println("=====");
    System.out.println("Paso la prueba de promedios?: " + (pasoPromedios ? "Sí" : "NO"));
    System.out.println("Paso la prueba de frecuencias?: " + (pasoFrecuencias ? "Sí" : "NO"));
    System.out.println("Paso la prueba de póquer?: " + (pasoPoquer ? "Sí" : "NO"));
    System.out.println("Paso la prueba de corridas?: " + (pasoCorridas ? "Sí" : "NO"));

    // Guarda los parámetros en el archivo de texto si pasa las pruebas
    if (pasoPromedios && pasoFrecuencias && pasoPoquer && pasoCorridas) {
        guardarParametrosAprobados("RSU1P6E1.txt", semilla, a, b, M);
    }
}

public static boolean evaluarPromedios(List<Double> numeros) {
    System.out.println("\n>>> 1. PRUEBA DE LOS PROMEDIOS");
    int N = numeros.size();
    double suma = 0;
    for (double num : numeros) suma += num;
    double promedio = suma / N;

    double z0 = (promedio - 0.5) * Math.sqrt(12 * N);
    double zCritico = 1.96;

    System.out.printf("Promedio: %.5f | Z0: %.5f | Limite: +-.2f%n", promedio, z0, zCritico);
    boolean pasa = Math.abs(z0) <= zCritico;
    System.out.printf("CONCLUSIÓN: Como |Z0| = %.4f <= %.2f, %s.%n", Math.abs(z0), zCritico,
        pasa ? "no se rechaza H0 (Pasa)" : "se rechaza H0 (Falla)");
    return pasa;
}
}

```

```

public static boolean evaluarFrecuencias(List<Double> numeros) {
    System.out.println("\n>>> 2. PRUEBA DE FRECUENCIAS (5 SUBINTERVALOS)");
    int N = numeros.size();
    int k = 5;
    int[] fo = new int[k];
    double fe = (double) N / k; // 4096 / 5 = 819.2

    for (double num : numeros) {
        int intervalo = (int) (num * k);
        if (intervalo >= k) intervalo = k - 1;
        fo[intervalo]++;
    }

    double chi0 = 0;
    for (int i = 0; i < k; i++) {
        chi0 += Math.pow(fo[i] - fe, 2) / fe;
    }

    double chiCritico = 9.488; // 4 grados de libertad
    System.out.printf(" Chi-cuadrada calculada: %.5f | Valor critico: %.3f\n", chi0, chiCritico);
    boolean pasa = chi0 <= chiCritico;
    System.out.printf(" CONCLUSIÓN: Como Chi0^2 = %.4f <= %.3f, %s.%n", chi0, chiCritico,
        pasa ? "no se rechaza H0 (Pasa)" : "se rechaza H0 (Falla)");
    return pasa;
}

public static boolean evaluarPoquer(List<Double> numeros) {
    System.out.println("\n>>> 3. PRUEBA DE PÓQUER (5 DECIMALES)");
    int N = numeros.size();
    int[] fo = new int[7]; // TD, 1P, 2P, T, FH, P, Q

    for (double num : numeros) {
        String str = String.format("%.5f", num);
        String decimales = str.substring(str.indexOf(".") + 1);
        if (decimales.length() > 5) decimales = decimales.substring(0, 5);

        Map<Character, Integer> conteo = new HashMap<>();
        for (char c : decimales.toCharArray()) {
            conteo.put(c, conteo.getOrDefault(c, 0) + 1);
        }

        List<Integer> valores = new ArrayList<>(conteo.values());
        Collections.sort(valores, Collections.reverseOrder());

        if (valores.get(0) == 5) fo[6]++; // Quintilla
        else if (valores.get(0) == 4) fo[5]++; // Póquer
        else if (valores.get(0) == 3 && valores.get(1) == 2) fo[4]++; // Full House
        else if (valores.get(0) == 3) fo[3]++; // Tercia
        else if (valores.get(0) == 2 && valores.get(1) == 2) fo[2]++; // Dos Pares
        else if (valores.get(0) == 2) fo[1]++; // Un Par
        else fo[0]++; // Todos Diferentes
    }

    // Probabilidades teóricas fijas para 5 decimales independientes
    double[] prob = {0.3024, 0.5040, 0.1080, 0.0720, 0.0090, 0.0045, 0.0001};
    double chi0 = 0;

    // Agrupamiento dinámico si FE < 5 (FH, P, Q se unen por seguridad matemática)
    double foAcum = 0, feAcum = 0;

    System.out.printf(" | %-18s | %-10s | %-10s |%n", "Mano", "FO", "FE");
    String[] manos = {"Todos Dif (TD)", "Un Par (1P)", "Dos Pares (2P)", "Tercia (T)", "Full House", "Poquer (P)", "Quintilla (Q)"};

    for (int i = 0; i < 7; i++) {
        double fe_i = N * prob[i];
        System.out.printf(" | %-18s | %-10d | %-10.2f |%n", manos[i], fo[i], fe_i);

        if (i < 4) {
            chi0 += Math.pow(fo[i] - fe_i, 2) / fe_i;
        } else {
            foAcum += fo[i];
            feAcum += fe_i;
        }
    }
}

```

```

        feAcum += fe_i;
    }
}
chi0 += Math.pow(foAcum - feAcum, 2) / feAcum; // Añade categoría agrupada

double chiCritico = 9.488; // 4 grados de libertad después de agrupar categorías
System.out.printf("    Chi-cuadrada calculada (con agrupamiento): %.5f | Valor critico: %.3f%n", chi0, chiCritico);
boolean pasa = chi0 <= chiCritico;
System.out.printf("    CONCLUSIÓN: Como Chi0^2 = %.4f <= %.3f, %s.%n", chi0, chiCritico,
    pasa ? "no se rechaza H0 (Pasa)" : "se rechaza H0 (Falla)");
return pasa;
}

public static boolean evaluarCorridas(List<Double> numeros) {
    System.out.println("\n>>> 4. PRUEBA DE LAS CORRIDAS (ARRIBA Y ABAJO)");
    int N = numeros.size();
    List<Integer> sec = new ArrayList<>();

    for (int i = 0; i < N - 1; i++) {
        sec.add(numeros.get(i) < numeros.get(i + 1) ? 0 : 1);
    }

    int fo_1 = 0, fo_2 = 0, fo_MayorIgual3 = 0;
    int longitudActual = 1;
    int totalCorridas = 0;

    for (int i = 0; i < sec.size() - 1; i++) {
        if (sec.get(i).equals(sec.get(i + 1))) {
            longitudActual++;
        } else {
            if (longitudActual == 1) fo_1++;
            else if (longitudActual == 2) fo_2++;
            else fo_MayorIgual3++;
            totalCorridas++;
            longitudActual = 1;
        }
    }

    if (longitudActual == 1) fo_1++;
    else if (longitudActual == 2) fo_2++;
    else fo_MayorIgual3++;
    totalCorridas++;

    double totalEsperadoEC = (2.0 * N - 1.0) / 3.0;
    double fe_1 = (2.0 * 5 * N - 7) / 24.0; // Simplificación de fórmula para i=1 con N grande
    double fe_2 = (2.0 * 11 * N - 14) / 120.0; // Simplificación para i=2
    double fe_MayorIgual3 = totalEsperadoEC - fe_1 - fe_2;

    double t1 = Math.pow(fo_1 - fe_1, 2) / fe_1;
    double t2 = Math.pow(fo_2 - fe_2, 2) / fe_2;
    double t3 = Math.pow(fo_MayorIgual3 - fe_MayorIgual3, 2) / fe_MayorIgual3;
    double chi0 = t1 + t2 + t3;

    double chiCritico = 5.99; // 2 grados de libertad
    System.out.printf("    Total Corridas (h): %d | Chi-cuadrada calculada: %.5f | Valor critico: %.2f%n",
        totalCorridas, chi0, chiCritico);
    boolean pasa = chi0 <= chiCritico;
    System.out.printf("    CONCLUSIÓN: Como Chi0^2 = %.4f <= %.2f, %s.%n", chi0, chiCritico,
        pasa ? "no se rechaza H0 (Pasa)" : "se rechaza H0 (Falla)");
    return pasa;
}

public static void guardarParametrosAprobados(String ruta, int sem, int a, int b, int m) {
    try (FileWriter fw = new FileWriter(ruta); PrintWriter pw = new PrintWriter(fw)) {
        pw.println("=====");
        pw.println("    REGISTRO DE PARÁMETROS CONFIGURADOS COMPATIBLES (APROBADOS)");
        pw.println("=====");
        pw.println("Los siguientes parametros superaron exitosamente las 4 pruebas");
        pw.println("estadísticas de uniformidad e independencia (Alpha = 0.05):");
        pw.println("-> Semilla Inicial (X0): " + sem);
        pw.println("-> Constante Multiplicativa (a): " + a);
        pw.println("-> Constante Aditiva (b): " + b);
        pw.println("-> Modulo (M): " + m);
        pw.println("-> Estatus del Generador: CERTIFICADO / COMPATIBLE");
        pw.println("=====");
    }
}

```

```

        pw.println("=====");
        System.out.println("\n[ÉXITO] Archivo '" + ruta + "' creado con los parámetros auditados.");
    } catch (IOException e) {
        System.out.println("Error al escribir archivo: " + e.getMessage());
    }
}
}

```

3.1.2 CÓDIGO SIN NOTAS (TEXTO)

```

package SIMULACION;

import java.io.FileWriter;

import java.io.IOException;

import java.io.PrintWriter;

import java.util.ArrayList;

import java.util.Collections;

import java.util.HashMap;

import java.util.List;

import java.util.Map;

public class SU1P6E1 {

    public static void main(String[] args) {

        // 1. Parámetros de la Práctica 4 (Periodo Completo de 4096)

        int semilla = 7326;

        int a = 205;    // Nuevo multiplicador con alta entropía

        int b = 1399;   // Nueva constante aditiva prima relativa

        int M = 4096;

        List<Double> numeros = new ArrayList<>();

        int actual = semilla;

        // Generación de la serie de 4096 números normalizados entre 0 y 1

        for (int i = 0; i < M; i++) {

```

```

        int siguiente = (a * actual + b) % M;

        numeros.add((double) siguiente / M);

        actual = siguiente;
    }

    System.out.println("=====
=====");

    System.out.println("===   PRÁCTICA 6: IMPLEMENTACIÓN DE PRUEBAS ESTADÍSTICAS
MASIVAS   ===");

    System.out.println("=====
=====");

    System.out.println("-> Procesando " + M + " números pseudoaleatorios generados...");

    // Ejecución de las 4 pruebas estadísticas requeridas

    boolean pasoPromedios = evaluarPromedios(numeros);

    boolean pasoFrecuencias = evaluarFrecuencias(numeros);

    boolean pasoPoquer = evaluarPoquer(numeros);

    boolean pasoCorridas = evaluarCorridas(numeros);

    System.out.println("\n=====
=====");

    System.out.println("===           RESUMEN DE AUDITORÍA           ===");

    System.out.println("=====
=====");

    System.out.println("¿Paso la prueba de promedios?: " + (pasoPromedios ? "Sí" : "NO"));

    System.out.println("¿Paso la prueba de frecuencias?: " + (pasoFrecuencias ? "Sí" : "NO"));

    System.out.println("¿Paso la prueba de póquer?: " + (pasoPoquer ? "Sí" : "NO"));

    System.out.println("¿Paso la prueba de corridas?: " + (pasoCorridas ? "Sí" : "NO"));

```



```

// Guarda los parámetros en el archivo de texto si pasa las pruebas
if (pasoPromedios && pasoFrecuencias && pasoPoquer && pasoCorridas) {
    guardarParametrosAprobados("RSU1P6E1.txt", semilla, a, b, M);
}
}

public static boolean evaluarPromedios(List<Double> numeros) {
    System.out.println("\n>>> 1. PRUEBA DE LOS PROMEDIOS");
    int N = numeros.size();
    double suma = 0;
    for (double num : numeros) suma += num;
    double promedio = suma / N;

    double z0 = (promedio - 0.5) * Math.sqrt(12 * N);
    double zCritico = 1.96;

    System.out.printf(" Promedio: %.5f | Z0: %.5f | Limite: +-.2f%\n", promedio, z0, zCritico);
    boolean pasa = Math.abs(z0) <= zCritico;
    System.out.printf(" CONCLUSIÓN: Como |Z0| = %.4f <= %.2f, %s.%n", Math.abs(z0), zCritico,
        pasa ? "no se rechaza H0 (Pasa)" : "se rechaza H0 (Falla)");
    return pasa;
}

public static boolean evaluarFrecuencias(List<Double> numeros) {
    System.out.println("\n>>> 2. PRUEBA DE FRECUENCIAS (5 SUBINTERVALOS)");
    int N = numeros.size();
    int k = 5;

```

```

int[] fo = new int[k];

double fe = (double) N / k; // 4096 / 5 = 819.2

for (double num : numeros) {
    int intervalo = (int) (num * k);
    if (intervalo >= k) intervalo = k - 1;
    fo[intervalo]++;
}

double chi0 = 0;
for (int i = 0; i < k; i++) {
    chi0 += Math.pow(fo[i] - fe, 2) / fe;
}

double chiCritico = 9.488; // 4 grados de libertad
System.out.printf(" Chi-cuadrada calculada: %.5f | Valor critico: %.3f%n", chi0, chiCritico);
boolean pasa = chi0 <= chiCritico;
System.out.printf(" CONCLUSIÓN: Como Chi0^2 = %.4f <= %.3f, %s.%n", chi0, chiCritico,
    pasa ? "no se rechaza H0 (Pasa)" : "se rechaza H0 (Falla)");
return pasa;
}

public static boolean evaluarPoquer(List<Double> numeros) {
    System.out.println("\n>>> 3. PRUEBA DE PÓQUER (5 DECIMALES)");
    int N = numeros.size();
    int[] fo = new int[7]; // TD, 1P, 2P, T, FH, P, Q

    for (double num : numeros) {

```

```

String str = String.format("%.5f", num);

String decimales = str.substring(str.indexOf(".") + 1);

if (decimales.length() > 5) decimales = decimales.substring(0, 5);


Map<Character, Integer> conteo = new HashMap<>();

for (char c : decimales.toCharArray()) {
    conteo.put(c, conteo.getOrDefault(c, 0) + 1);
}


List<Integer> valores = new ArrayList<>(conteo.values());

Collections.sort(valores, Collections.reverseOrder());


if (valores.get(0) == 5) fo[6]++; // Quintilla
else if (valores.get(0) == 4) fo[5]++; // Póquer
else if (valores.get(0) == 3 && valores.get(1) == 2) fo[4]++; // Full House
else if (valores.get(0) == 3) fo[3]++; // Tercia
else if (valores.get(0) == 2 && valores.get(1) == 2) fo[2]++; // Dos Pares
else if (valores.get(0) == 2) fo[1]++; // Un Par
else fo[0]++; // Todos Diferentes
}


// Probabilidades teóricas fijas para 5 decimales independientes
double[] prob = {0.3024, 0.5040, 0.1080, 0.0720, 0.0090, 0.0045, 0.0001};

double chi0 = 0;


// Agrupamiento dinámico si FE < 5 (FH, P, Q se unen por seguridad matemática)
double foAcum = 0, feAcum = 0;

```

```

        System.out.printf(" | %-18s | %-10s | %-10s |%n", "Mano", "FO", "FE");

        String[] manos = {"Todos Dif (TD)", "Un Par (1P)", "Dos Pares (2P)", "Tercia (T)", "Full House",
        "Poquer (P)", "Quintilla (Q)"};

        for (int i = 0; i < 7; i++) {

            double fe_i = N * prob[i];

            System.out.printf(" | %-18s | %-10d | %-10.2f |%n", manos[i], fo[i], fe_i);

            if (i < 4) {

                chi0 += Math.pow(fo[i] - fe_i, 2) / fe_i;

            } else {

                foAcum += fo[i];

                feAcum += fe_i;

            }

        }

        chi0 += Math.pow(foAcum - feAcum, 2) / feAcum; // Añade categoría agrupada

        double chiCritico = 9.488; // 4 grados de libertad después de agrupar categorías

        System.out.printf(" Chi-cuadrada calculada (con agrupamiento): %.5f | Valor critico: %.3f%n",
        chi0, chiCritico);

        boolean pasa = chi0 <= chiCritico;

        System.out.printf(" CONCLUSIÓN: Como Chi0^2 = %.4f <= %.3f, %s.%n", chi0, chiCritico,
        pasa ? "no se rechaza H0 (Pasa)" : "se rechaza H0 (Falla)");

        return pasa;

    }

    public static boolean evaluarCorridas(List<Double> numeros) {

        System.out.println("\n>>> 4. PRUEBA DE LAS CORRIDAS (ARRIBA Y ABAJO)");

        int N = numeros.size();

```

```

List<Integer> sec = new ArrayList<>();

for (int i = 0; i < N - 1; i++) {
    sec.add(numeros.get(i) < numeros.get(i + 1) ? 0 : 1);
}

int fo_1 = 0, fo_2 = 0, fo_MayorIgual3 = 0;
int longitudActual = 1;
int totalCorridas = 0;

for (int i = 0; i < sec.size() - 1; i++) {
    if (sec.get(i).equals(sec.get(i + 1))) {
        longitudActual++;
    } else {
        if (longitudActual == 1) fo_1++;
        else if (longitudActual == 2) fo_2++;
        else fo_MayorIgual3++;
        totalCorridas++;
        longitudActual = 1;
    }
}

if (longitudActual == 1) fo_1++;
else if (longitudActual == 2) fo_2++;
else fo_MayorIgual3++;
totalCorridas++;

double totalEsperadoEC = (2.0 * N - 1.0) / 3.0;

double fe_1 = (2.0 * 5 * N - 7) / 24.0; // Simplificación de fórmula para i=1 con N grande

```

```

double fe_2 = (2.0 * 11 * N - 14) / 120.0; // Simplificación para i=2

double fe_MayorIgual3 = totalEsperadoEC - fe_1 - fe_2;

double t1 = Math.pow(fo_1 - fe_1, 2) / fe_1;

double t2 = Math.pow(fo_2 - fe_2, 2) / fe_2;

double t3 = Math.pow(fo_MayorIgual3 - fe_MayorIgual3, 2) / fe_MayorIgual3;

double chi0 = t1 + t2 + t3;

double chiCritico = 5.99; // 2 grados de libertad

System.out.printf("    Total Corridas (h): %d | Chi-cuadrada calculada: %.5f | Valor critico:
%.2f%n",

    totalCorridas, chi0, chiCritico);

boolean pasa = chi0 <= chiCritico;

System.out.printf("    CONCLUSIÓN: Como Chi0^2 = %.4f <= %.2f, %s.%n", chi0, chiCritico,

    pasa ? "no se rechaza H0 (Pasa)" : "se rechaza H0 (Falla)");

return pasa;
}

public static void guardarParametrosAprobados(String ruta, int sem, int a, int b, int m) {

    try (FileWriter fw = new FileWriter(ruta); PrintWriter pw = new PrintWriter(fw)) {

        pw.println("=====");

        pw.println("    REGISTRO DE PARÁMETROS CONFIGURADOS COMPATIBLES
(APROBADOS)");

        pw.println("=====");

        pw.println("Los siguientes parametros superaron exitosamente las 4 pruebas");

        pw.println("estadisticas de uniformidad e independencia (Alpha = 0.05):");

        pw.println("-> Semilla Inicial (X0): " + sem);

        pw.println("-> Constante Multiplicativa (a): " + a);

        pw.println("-> Constante Aditiva (b): " + b);

```

```
        pw.println("-> Modulo (M): " + m);  
        pw.println("-> Estatus del Generador: CERTIFICADO / COMPATIBLE");  
        pw.println("=====");  
        System.out.println("\n[ÉXITO] Archivo " + ruta + " creado con los parámetros auditados.");  
    } catch (IOException e) {  
        System.out.println("Error al escribir archivo: " + e.getMessage());  
    }  
}  
}
```

3.2 EJECUCIÓN

3.2.1 RUN

```
=====
===      PRÁCTICA 6: IMPLEMENTACIÓN DE PRUEBAS ESTADÍSTICAS MASIVAS      ===
=====

-> Procesando 4096 números pseudoaleatorios generados...

>>> 1. PRUEBA DE LOS PROMEDIOS
    Promedio: 0.49988 | Z0: -0.02706 | Limite: +-1.96
    CONCLUSIÓN: Como |Z0| = 0.0271 <= 1.96, no se rechaza H0 (Pasa).

>>> 2. PRUEBA DE FRECUENCIAS (5 SUBINTERVALOS)
    Chi-cuadrada calculada: 0.00098 | Valor critico: 9.488
    CONCLUSIÓN: Como Chi0^2 = 0.0010 <= 9.488, no se rechaza H0 (Pasa).

>>> 3. PRUEBA DE POQUER (5 DECIMALES)
    | Mano          | FO          | FE          |
    | Todos Dif (TD) | 1238        | 1238.63     |
    | Un Par (1P)    | 2104        | 2064.38     |
    | Dos Pares (2P) | 400         | 442.37      |
    | Tercia (T)     | 303         | 294.91      |
    | Full House     | 38          | 36.86       |
    | Poquer (P)     | 12          | 18.43       |
    | Quintilla (Q)  | 1           | 0.41        |
    Chi-cuadrada calculada (con agrupamiento): 5.43768 | Valor critico: 9.488
    CONCLUSIÓN: Como Chi0^2 = 5.4377 <= 9.488, no se rechaza H0 (Pasa).

>>> 4. PRUEBA DE LAS CORRIDAS (ARRIBA Y ABAJO)
    Total Corridas (h): 2750 | Chi-cuadrada calculada: 0.86801 | Valor critico: 5.99
    CONCLUSIÓN: Como Chi0^2 = 0.8680 <= 5.99, no se rechaza H0 (Pasa).

=====
===      RESUMEN DE AUDITORÍA      ===
=====

❖ Paso la prueba de promedios?: S
❖ Paso la prueba de frecuencias?: S
❖ Paso la prueba de pquer?: S
❖ Paso la prueba de corridas?: S

[EXITO] Archivo 'RSU1P6E1.txt' creado con los parámetros auditados.
```


3.2.2 ARCHIVO

```
=====
REGISTRO DE PARÁMETROS CONFIGURADOS COMPATIBLES (APROBADOS)
=====
Los siguientes parametros superaron exitosamente las 4 pruebas
estadísticas de uniformidad e independencia (Alpha = 0.05):
-> Semilla Inicial (X0): 7326
-> Constante Multiplicativa (a): 205
-> Constante Aditiva (b): 1399
-> Modulo (M): 4096
-> Estatus del Generador: CERTIFICADO / COMPATIBLE
=====
```

4 CONCLUSIONES

Al culminar la presente práctica, se implementó de forma robusta un entorno automatizado de auditoría estadística masiva en el lenguaje Java. El experimento demostró de forma contundente que cumplir con el Teorema de Periodo Completo es una condición necesaria pero no suficiente para catalogar un generador congruencial como apto para la simulación profesional. Como se constató en la iteración inicial con el juego de constantes (a=17, b=9), la serie generó números únicos pero falló severamente en su criterio de independencia en la prueba de corridas ($X_0^2 = 24.87 > 5.99$). Tras ejecutar un rediseño modular riguroso de parámetros lógicos utilizando las constantes de alta entropía a = 205 y b = 1399, se logró que la secuencia de 4096 datos se comportara de forma estadísticamente homogénea, aprobando satisfactoriamente los límites críticos de uniformidad y distribución aleatoria por dígitos (Póquer). La persistencia física lograda mediante el archivo externo RSU1P6E1.txt certifica un conjunto seguro de parámetros de simulación estocástica para futuras aplicaciones con el método de Montecarlo.

5 GITHUB

<https://github.com/anamarbaku999-cell/SIMULACION>